Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · Decorators

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · Decorators
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

Decorators

1 Decorators

Study focus: Decorators. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000066 · EV0000067

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Python for computational problem solving.

Key details

- In today's class, we are going to be looking at the concept called as decorators.
- In order to understand decorators, there are a few other things that are required.
- And just to remind you, these things have already been the concepts that I'm going to be doing now have already been covered in the previous videos.
- I kindly request you to check.
- The ones that are required are.
- Will be executed by me now so that it is going to be easy for you to understand the concept of decorators.
- Let me create a function called as.
- And what I really want to do with this is to print this message.
- Let it be as simple as possible.
- I will also do something like this, F2 is equal to F1, right?
- The message that I'm trying to pass now is call as decorator.
- At this point.
- F1 and F2 are pointing to the same object internally.
- in order to make our life easy.
- So what is the next thing that I'm going to be doing is when we passed the arguments to a function, we passed some message and all.
- Function as an argument at this particular point.
- Let me define a function increment.
- Hope it is okay.

- So now in this case I will say a return x plus one because the incrementation operation and I will write yet another function called as decrement.
- And for the decrement also, I will pass the value, any value that of your choice.
- Just look at what I'm going to be doing now.
- Fine and I will retain its return value in this case.
- And then followed by this.
- This is fine for you.
- Hope there are no confusions here.
- This is clear?
- operations of let me call increment and the value that I will give is ten.

Example or application from the lecture

- Let's quickly execute this.
- Let's quickly execute this and check it out.
- Let's let me comment this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000066 · EV0000067 · EV0000068 · +1 source units in pack.json

Core explanation · 05:00–09:59

We quickly execute.

Key details

- And if you see there are no errors, and we are getting the result, which is 11.
- Body is processed, sorry, header is processed every time, sorry, it's a header.
- The function call is for this particular function.
- An increment is yet another function.
- Is this fine?
- And this x fine will get the value called as 10.
- Fine, because this function is increment and x value is 10.
- What it gets returned, it returns x plus 1.
- X plus 1.
- You are returning this particular result.
- You are returning it to this place where it was called.
- It has returned 11, and then you are trying to print a value called as 11.
- It's all about just understanding how things are going on, nothing much, just.
- Define one more function which is going to be called.
- Fine, a function called.
- Then followed by I will define one more function, function returned.
- Hope this is fine for you.
- Is it confusing?
- The question mark because I don't know.
- I'm unable to identify whether you're able to understand or not.
- And I will say return.

- Function underscore returned.
- Hope this is fine for you.
- I'll just come back here and then say something new is equal to the function that is called.
- What is the output that it gives us?
- And now I will say new of this.
- That's fine.
- These things have already been done up to you.
- You're calling a function, I get into the function.
- The moment the headers are processed, what is the next thing that I have to do?
- I'll have to wait for the function to be called.
- And then I will process this particular function.
- Will you get it into the body of this function return?
- No, I won't get it until it has been called.
- The object has been returned.
- Where it is returned, it is returned.
- To this particular point.
- It has returned the address where this particular function is available.
- Where this program basically illustrates that a function can return another function.
- Our actual concept is decorator.

Example or application from the lecture

- If there are any errors in this program, let's try to identify what is an error in this case.
- Let's find out.
- A function can return another function is what we have done or we have understood in this particular example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000070 · EV0000071 · EV0000072 · +1 source units in pack.json

Core explanation · 10:00–14:59

It takes one more function.

Key details

- Some other functionality.
- Let me repeat.
- It adds some other functionality.
- Hopefully it is clear and it reser returns the result.
- Decorator is a function, it takes another function, it adds some other functionality with the existing functionality and gives you the result.
- It is as similar as you wrapping a gift.
- You have a gift inside a box.
- What you basically do is you try to decorate your gift box in such a way that it is so nice.
- Whatever you try to do, you're trying to do it for your gift box and the object within that gift box remains the same.

- This is fine.
- Is this fine?
- You add some functionality to the existing code.
- The original code can be reused again and again according to your requirement because you are trying to use the same function, but add a few other things around it.
- That's called as a decorator.
- I say define make a pretty [ASR low confidence; check audio].
- [ASR low confidence; check audio].
- We do this.
- I look nice after decoration.
- I look nice after decoration.
- This is my function.
- I'm going to be calling this function.
- We'll see return function.
- This is not confusing you.
- I think then.
- What do I do?
- We'll try to call the function.
- Find out are there any errors in my program.
- Header gets processed, body never gets in, this header gets processed, but I am trying to call this function.
- Just to find out are there any errors in our program.
- And it says that I look very simple.
- Let me comment or let me remove this so that you will not have any confusions for you.
- So next thing that I'm going to be doing is pretty.

Example or application from the lecture

- Let me remind you, I'm taking the same example which is available on the internet itself because this example is very nice to understand.
- This example is from the internet material itself.
- Let me repeat, this is from the internet example itself.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000074 · EV0000075 · EV0000076 · +1 source units in pack.json

Core explanation · 15:00–19:59

And for this I possess an ordinary.

Key details

- I pass on ordinary.
- This is fine.
- Hope this is fine.
- Don't get confused, so I'll see pretty.
- Don't worry, I will show it on Python Tutor in order to understand.

- I look very simple now.
- You can change the statements.
- I look very simple now.
- After decoration, I look beautiful, all those things you can just do the changes and then work it out.
- What are we trying to do?
- We are trying to call make pretty.
- This is a place where decoration is taking place.
- Is this fine?
- This is a place where decoration is taking place.
- Make pretty, you're calling a function, and then you call ordinary.
- Is this fine?
- At this point, what does function contains?
- Your function contains ordinary.
- This is a point where your ordinary function has been called.
- Is this fine?
- Get a return address.
- The return address is present at pretty.
- I am calling the function.
- Pritty has inner function.
- The moment you call the inner function, what I'm doing trying to do, I look nice after decoration.
- When you and then.
- And followed by you have a function here.
- Previously I was looking very simple.
- After decoration that took place, I look beautiful.
- There is a few other things.
- We'll I'll just quickly do this.
- Let me eliminate these two sentences.
- I'm not going to be using this.
- I'll just call ordinary.
- This is fine.
- Nothing else I'm calling.
- The moment I call ordinary, it becomes like I look very simple.
- But now did I not eliminate the concept of decorator here?
- Yes, I have eliminated.
- An ordinary function pretty the make a pretty function.
- And I'm making this as what?
- A prettier one.
- This is fine.
- The last two statements that I eliminated, which is equal to these two.
- One second, give me time.
- Otherwise it just becomes an header processing without a function being called.
- And if you can see I look nice after decoration, I was looking very simple.

- I look very simple now.
- Previously, if you have seen our output remained the same.
- The last two statements I eliminated and this has been.
- Be placed with at the rate symbol followed by make written.
- Fourteen lines are there.
- Function was processed.
- Define the inner function.
- The moment you return, what are you trying to do?

Example or application from the lecture

- Let's quickly get started.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000078 · EV0000079 · EV0000080 · +1 source units in pack.json

Core explanation · 20:00–24:59

Where it has returned, it has returned to ordinary.

Key details

- Then now go back to where it has called.
- Then I look nice after decoration.
- And you're calling an ordinary function here.
- You come back here and then print I look very simple now.
- Hope you have understood.
- in the theory class when we do we will be looking into deeper into these things in order to understand this concept in a better way.
- Hope you are all fine over this.
- I request you to check it take out few more programs and work on the same.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000082 · EV0000083 · EV0000084 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Decorators transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Decorators transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Decorators transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9998; only the JSON segmentation supplies the displayed timestamps.
Decorators transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/011_Decorators__6b4052527d795156.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.

6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00026	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...
SRC00027	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00028	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...
SRC00029	5 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0022. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0023 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0067. Sidecar variants compare...

C Revision and retrieval practice